

# **Pointers and Dynamic Memory Management**

**Mao © 2023**

# Outline



- **Pointer**
- **Using const with Pointers**
- **Defining Synonymous Types Using the typedef Keyword**
- **Pointers and Arrays**
- **Pointers and C-strings**
- **Passing Pointer Arguments in a Function Call**
- **Returning a Pointer from Functions**
- **Useful Array Functions**
- **Creating and Accessing Dynamic Objects**
- **The this Pointer**

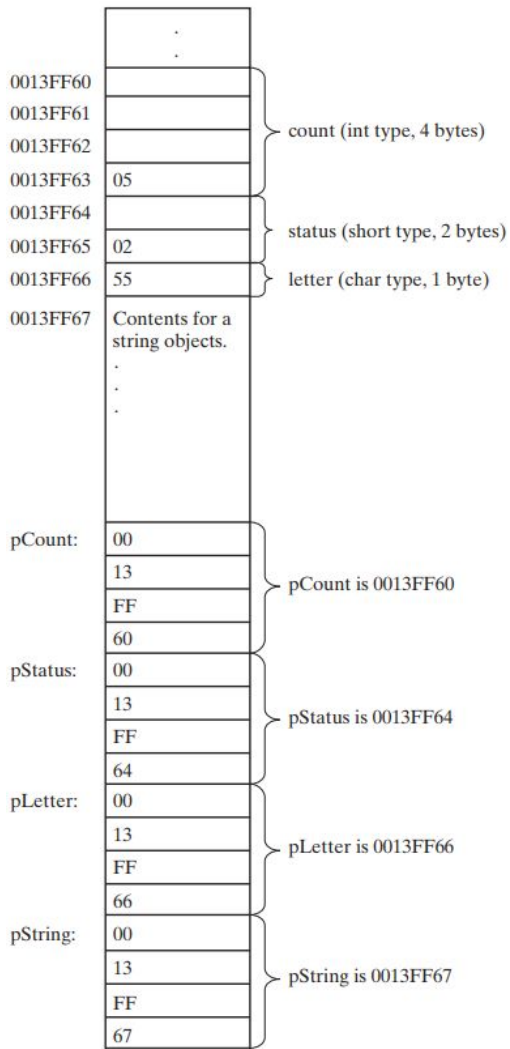
# Pointer



A pointer variable (simply called pointer) **holds the memory address**. Through the pointer, **the dereference operator \*** can be used to access **the actual value at a specific memory location**.

Normally, a variable contains a data value—e.g., an integer, a floating-point value, and a character. However, a pointer contains the memory address of a variable that in turn contains a data value.

# Pointer



```
int count = 5;
```

```
short status = 2;
```

```
char letter = 'A';
```

```
string s = "ABC";
```

```
int* pCount = &count;
```

```
short* pStatus = &status;
```

```
char* pLetter = &letter;
```

```
string* pString = &s;
```

```
pCount = &count;
```

**TestPointer.cpp**

**&: address operator**

**&count means the address of count**

**\*: dereferencing operator**

**\*pCount means the value pointed by pCount is assigned to v**

# Pointer

A pointer can be initialized when it is declared or by using an assignment statement. However, if you assign an address to a pointer, the syntax is

**pCount = &count; // Correct**

rather than

**\*pCount = &count; // Wrong**

# Pointer

Referencing a value through a pointer is often called indirection. The syntax for referencing a value from a pointer is

**\*pointer**

For example, you can increase count using

**count++; // Direct reference**

or

**(\*pCount)++; // Indirect reference**

# Pointer

A pointer variable is declared with a type such as `int` or `double`. You have to assign the address of the variable of the same type. It is a syntax error if the type of the variable does not match the type of the pointer. For example, the following code is wrong:

```
int area = 1;
```

```
double* pArea = &area; // Wrong
```

Note that pointers **can be compared using relational operators** (`==`, `!=`, `<`, `<=`, `>`, `>=`) to determine their order.

# Pointer

`int i = 15, j, *p, *q;`

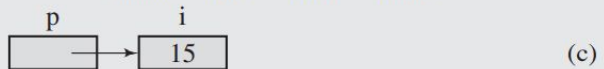
| i    | j    | p    | q    |
|------|------|------|------|
| 15   | ?    | ?    | ?    |
| 1080 | 1082 | 1084 | 1086 |

(a)

`p = &i;`

| i    | j    | p    | q    |
|------|------|------|------|
| 15   | ?    | 1080 | ?    |
| 1080 | 1082 | 1084 | 1086 |

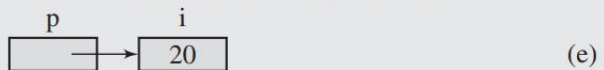
(b)



`*p = 20;`

| i    | j    | p    | q    |
|------|------|------|------|
| 20   | ?    | 1080 | ?    |
| 1080 | 1082 | 1084 | 1086 |

(d)



`j = 2 * *p;`

| i    | j    | p    | q    |
|------|------|------|------|
| 20   | 40   | 1080 | ?    |
| 1080 | 1082 | 1084 | 1086 |

(f)

`q = &i;`

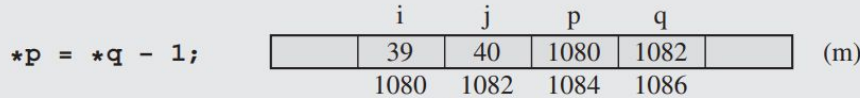
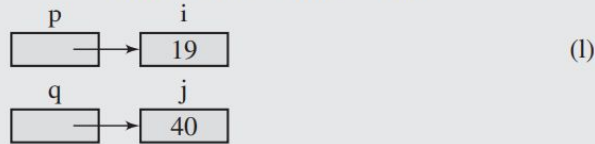
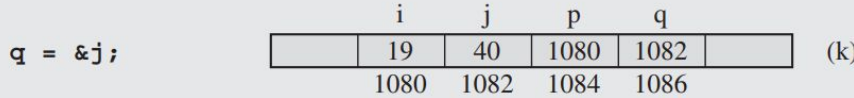
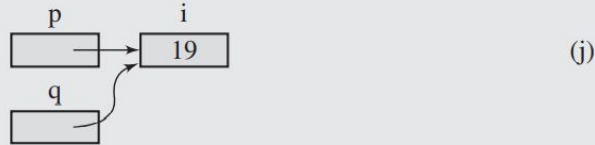
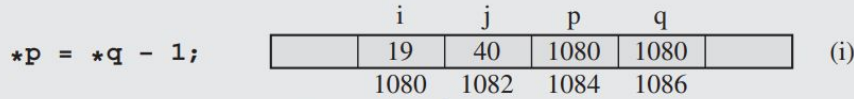
| i    | j    | p    | q    |
|------|------|------|------|
| 20   | 40   | 1080 | 1080 |
| 1080 | 1082 | 1084 | 1086 |

(g)



(continuing)

# Pointer



Changes of values after assignments are made using pointer variables. Note that (b) and (c) show the same situation, and so do (d) and (e), (g) and (h), (i) and (j), (k) and (l), and (m) and (n).

# Using const with Pointers

A constant pointer points to a constant memory location, but the actual value in the memory location can be changed. For example:

```
double radius = 5;
```

```
double* const p = &radius;
```

Here `p` is a constant pointer. It must be declared and initialized in the same statement. A new address cannot be assigned to `p` later. Though `p` is a constant, the data pointed to by `p` is not constant and can be changed. For example, the following statement changes `radius` to 10:

```
*p = 10;
```

# Defining Synonymous Types Using the typedef Keyword

A synonymous type can be defined using the typedef keyword.

The syntax to define a new synonym for an existing data type is as follows:

```
typedef existingType newType;
```

For example, the following statement defines integer as a synonym for int:

```
typedef int integer
```

Now an int variable can be declared using

```
integer value = 40;
```

# Defining Synonymous Types Using the typedef Keyword

The typedef declaration **does not create new data types**. It merely creates a synonym for a data type. This feature is useful to define a pointer type name to make the program easy to read. For example:

```
typedef int* intPointer;
```

An integer pointer variable can now be declared as follows:

```
intPointer p;
```

which is the same as

```
int* p;
```

# Defining Synonymous Types Using the typedef Keyword

One advantage of using a pointer type name is to avoid the errors involving missing asterisks, the following declaration is wrong to declare two pointer variables:

```
int* p1, p2; // p2 will just be an integer
```

A good way to avoid this error is to use the synonymous type **intPointer** as follows:

```
intPointer p1, p2;
```

# Pointers and Arrays

A C++ array name is actually a **constant pointer** to **the first element in the array**.

An array without a bracket and a subscript actually represents the starting address of the array. In this sense, an array is essentially a pointer. Suppose you declare an array of int values as follows:

```
int list[6] = {11, 12, 13, 14, 15, 16};
```

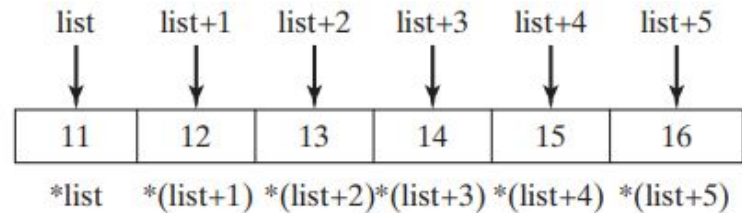
The following statement displays the starting address of the array:

```
cout << "The starting address of the array is " << list << endl;
```

# Pointers and Arrays

C++ allows you to access the elements in the array using the dereference operator. To access the first element, use `*list`. Other elements can be accessed using `*(list + 1)`, `*(list + 2)`, `*(list + 3)`, `*(list + 4)`, and `*(list + 5)`.

| <code>list[0]</code> | <code>list[1]</code> | <code>list[2]</code> | <code>list[3]</code> | <code>list[4]</code> | <code>list[5]</code> |
|----------------------|----------------------|----------------------|----------------------|----------------------|----------------------|
| 11                   | 12                   | 13                   | 14                   | 15                   | 16                   |



Note that `list` should be treated as a constant so that any attempt to assign a value to `list`, as in `list++`, is considered a compilation error.

# Pointers and Arrays

TestArrayPointer.cpp

To declare an int pointer p assigned with the address of the array.

```
int* p = list;
```

Note that we do not use the address operator (&) to assign the address of the array to the pointer, because **the name of the array is already the starting address of the array**. This line is equivalent to

```
int* p = &list[0];
```

Here, **&list[0]** represents the address of list[0].

# Pointers and Arrays

Arrays can be declared dynamically. To that end, pointer variables are used. For example, the assignment

```
int n;
```

```
cin>>n;
```

```
int * p = new int[n];
```

**//int list[n]; // No version of C++ allows a variable length array. Only C99 allows it. G++ allows it as an extension.**

If the array currently pointed to by p is no longer needed, it should be disposed of by the instruction

```
delete [] p; // not “delete p” here;
```

# Pointers and C-strings

**C-strings are often referred to as pointer-based strings**, because they can be conveniently accessed using pointers. For example, the following two declarations are both fine:

```
char city[7] = "Dallas"; // Option 1
```

```
char* pCity = "Dallas"; // Option 2
```

**Note that '\0' is the null termination character**. It marks the **end of the string**.

# Passing Pointer Arguments in a Function Call

There are two ways to pass arguments to a function in C++: pass-by-value and pass-by-reference.

A pointer argument **can be passed by value or by reference**. For example:

```
void f(int* p1, int* &p2)
```

which is equivalently to

```
typedef int* intPointer;
```

```
void f(intPointer p1, intPointer& p2)
```

# Passing Pointer Arguments in a Function Call

Consider invoking function `f(q1, q2)` with two pointers `q1` and `q2`:

- The pointer `q1` is passed to `p1` by value. So `*p1` and `*q1` point to the same contents.
  - If function `f` changes `*p1` (e.g., `*p1 = 20`), `*q1` is changed too.
  - However, if function `f` changes `p1` (e.g., `p1 = somePointerVariable`), `q1` is not changed.
- The pointer `q2` is passed to `p2` by reference. So `q2` and `p2` are now aliases. They are essentially the same.
  - If function `f` changes `*p2` (e.g., `*p2 = 20`), `*q2` is changed too.
  - If function `f` changes `p2` (e.g., `p2 = somePointerVariable`), `q2` is changed too.

**TestPointerArgument.cpp**

# Passing Pointer Arguments in a Function Call

An array parameter in a function can always be replaced using a pointer parameter. For example,

```
void m(int list[], int size)
```

can be replaced by

```
void m(int* list, int size)
```

```
void m(char c_string[])
```

can be replaced by

```
void m(char* c_string)
```

# Passing Pointer Arguments in a Function Call

If a value does not change, declare it `const` to prevent it from being accidentally modified.

```
void printArray(const int* list, const int size)
{
    for (int i = 0; i < size; i++)
        cout << list[i] << " ";
}
```

The `printArray` function declares an array parameter with constant data. This ensures that the contents of the array will not be changed.

# Returning a Pointer from Functions

A C++ function may return a pointer.

```
int* reverse(int* list, int size)
{
    for (int i = 0, j = size - 1; i < j; i++, j--)
    {
        // Swap list[i] with list[j]
        int temp = list[j];
        list[j] = list[i];
        list[i] = temp;
    }
    return list;
}
```

**ReverseArrayUsingPointer.cpp**

# Useful Array Functions

The `min_element`, `max_element`, `sort`, `random_shuffle`, and `find` functions can be used for arrays.

- the `min_element` and `max_element` functions are used to return **the pointer** to the minimal and maximal element in an array,
- the `sort` function to sort an array,
- the `random_shuffle` function to randomly shuffle an array,
- and the `find` function to find an element in an array.

All these functions use pointers in the arguments and in the return value.

`TestUsefulArrayFunctions.cpp`

# Creating and Accessing Dynamic Objects

To create an object dynamically, invoke the constructor for the object using the syntax

```
new ClassName(arguments).
```

To create an object using the **no-argument constructor** and assigns the object address to a pointer:

```
ClassName* pObj = new ClassName(); // or
```

```
ClassName* pObj = new ClassName;
```

If a class has a zero argument constructor, the following line creates a pointer pointing to an array of objects.

```
ClassName* pObj = new ClassName[5]; // pObj[0] is the first object
```

To create an object using the constructor **with arguments** and assigns the object address to a pointer.

```
ClassName* pObj = new ClassName(arguments);
```

# Creating and Accessing Dynamic Objects

For example,

```
// Create an object using the no-argument constructor
string* p = new string(); // or string* p = new string;
// Create an object using the constructor with arguments
string* p = new string("abcdefg");
```

To access object members via a pointer, dereference the pointer and use the dot (.) operator to object's members. For example,

```
string* p = new string("abcdefg");
cout << "The first three characters in the string are " << (*p).substr(0, 3) << endl;
cout << "The length of the string is " << (*p).length() << endl;
```

# Creating and Accessing Dynamic Objects

C++ also provides **a shorthand member selection operator** for accessing object members from a pointer: **arrow operator (->)**, which is a dash (-) immediately followed by the greaterthan (>) symbol.

For example,

```
cout << "The first three characters in the string are " << p->substr(0, 3) << endl;  
cout << "The length of the string is " << p->length() << endl;
```

The objects are destroyed when the program is terminated. To explicitly destroy an object, invoke

```
delete p;
```

# The **this** Pointer

The **this** pointer **points to the calling object itself**.

When a **data field name** is used as the **parameter name** in a set function for the data field, **a hidden data field can be accessed by using the this keyword**, which is a special built-in pointer that references the calling object.

```
// Construct a circle object
Circle::Circle(double radius)
{
    this->radius = radius; // or (*this).radius = radius;
}
```

# Reference



- **Revel for Introduction to C++ Programming and Data Structures by Daniel Liang, 5th edition, Pearson, 2022**